Secure Login System

Complete GitHub Project — Code + Setup + Security Details

This project is based on the supplied task image. It implements a secure login web app using Flask, Argon2 password hashing, input validation, parameterized SQLite queries, session management, logout, and optional TOTP-based two-factor authentication (2FA).

Feature	Implementation
Registration / Login	Flask routes with form validation
Password security	Argon2 password hashing
SQL injection protection	Parameterized SQLite queries (?)
Session management	Flask session + login_required decorator
Logout	session.clear()
Optional 2FA	TOTP using pyotp

1. Project Folder Structure

```
secure-login-system/
■
■■■ app.py
■■■ requirements.txt
■■■ README.md
■■■ .gitignore
■■■ users.db                # created automatically; do NOT upload
■
■■■ templates/
■   ■■■ base.html
■   ■■■ login.html
■   ■■■ register.html
■   ■■■ dashboard.html
■   ■■■ verify_2fa.html
■   ■■■ enable_2fa.html
■
■■■ static/
    ■■■ style.css
```

2. app.py

```
from flask import Flask, render_template, request, redirect, url_for, session, flash
import sqlite3
from argon2 import PasswordHasher
from argon2.exceptions import VerifyMismatchError
from functools import wraps
import pyotp
import os

app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY", "change-this-secret-key")
DATABASE = "users.db"
ph = PasswordHasher()

def get_db():
    conn = sqlite3.connect(DATABASE)
    conn.row_factory = sqlite3.Row
    return conn

def init_db():
    conn = get_db()
    conn.execute("""
        CREATE TABLE IF NOT EXISTS users (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            username TEXT UNIQUE NOT NULL,
            password_hash TEXT NOT NULL,
            totp_secret TEXT
        )
    """)
    conn.commit()
    conn.close()

def login_required(view):
    @wraps(view)
    def wrapped_view(*args, **kwargs):
        if "user_id" not in session:
            flash("Please log in first.", "error")
            return redirect(url_for("login"))
        return view(*args, **kwargs)
    return wrapped_view

@app.route("/")
def home():
    if "user_id" in session:
        return redirect(url_for("dashboard"))
    return redirect(url_for("login"))

@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")

        if len(username) < 3 or len(username) > 50:
            flash("Username must be 3-50 characters.", "error")
            return render_template("register.html")

        if len(password) < 8:
            flash("Password must contain at least 8 characters.", "error")
            return render_template("register.html")

        password_hash = ph.hash(password)

        try:
            conn = get_db()
            conn.execute(
                "INSERT INTO users (username, password_hash) VALUES (?, ?)",
```

```
                (username, password_hash)
            )
            conn.commit()
            conn.close()
            flash("Registration successful. Please log in.", "success")
            return redirect(url_for("login"))
        except sqlite3.IntegrityError:
            flash("Username already exists.", "error")

    return render_template("register.html")

@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        username = request.form.get("username", "").strip()
        password = request.form.get("password", "")

        conn = get_db()
        user = conn.execute(
            "SELECT * FROM users WHERE username = ?",
            (username,)
        ).fetchone()
        conn.close()

        if user:
            try:
                ph.verify(user["password_hash"], password)
                session.clear()
                session["pending_user_id"] = user["id"]

                if user["totp_secret"]:
                    return redirect(url_for("verify_2fa"))

                session["user_id"] = user["id"]
                session["username"] = user["username"]
                session.pop("pending_user_id", None)
                return redirect(url_for("dashboard"))
            except VerifyMismatchError:
                pass

        flash("Invalid username or password.", "error")

    return render_template("login.html")

@app.route("/2fa", methods=["GET", "POST"])
def verify_2fa():
    if "pending_user_id" not in session:
        return redirect(url_for("login"))

    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE id = ?",
        (session["pending_user_id"],)
    ).fetchone()
    conn.close()

    if not user or not user["totp_secret"]:
        return redirect(url_for("login"))

    if request.method == "POST":
        code = request.form.get("code", "").strip()

        if pyotp.TOTP(user["totp_secret"]).verify(code):
            session["user_id"] = user["id"]
            session["username"] = user["username"]
            session.pop("pending_user_id", None)
            return redirect(url_for("dashboard"))

        flash("Invalid 2FA code.", "error")

    return render_template("verify_2fa.html")

@app.route("/enable-2fa", methods=["GET", "POST"])
@login_required
def enable_2fa():
    conn = get_db()
    user = conn.execute(
        "SELECT * FROM users WHERE id = ?",
        (session["user_id"],)
    ).fetchone()

    if request.method == "POST":
        secret = request.form.get("secret", "").strip()
        if secret:
            conn.execute(
                "UPDATE users SET totp_secret = ? WHERE id = ?",
                (secret, session["user_id"])
            )
            conn.commit()
            conn.close()
            flash("2FA enabled.", "success")
            return redirect(url_for("dashboard"))
```

```
    secret = pyotp.random_base32()
    conn.close()
    return render_template("enable_2fa.html", secret=secret)

@app.route("/dashboard")
@login_required
def dashboard():
    return render_template("dashboard.html", username=session["username"])

@app.route("/logout")
def logout():
    session.clear()
    flash("You have been logged out.", "success")
    return redirect(url_for("login"))

if __name__ == "__main__":
    init_db()
    app.run(debug=True)
```

templates/base.html

```
<!doctype html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Secure Login System</title>
    <link rel="stylesheet" href="{{ url_for('static', filename='style.css') }}">
</head>
<body>
<div class="container">
    <h1>Secure Login System</h1>

    {% with messages = get_flashed_messages(with_categories=true) %}
      {% for category, message in messages %}
        <div class="message {{ category }}">{{ message }}</div>
      {% endfor %}
    {% endwith %}

    {% block content %}{% endblock %}
</div>
</body>
</html>
```

templates/login.html

```
{% extends "base.html" %}
{% block content %}
<h2>Login</h2>
<form method="POST">
    <label>Username</label>
    <input type="text" name="username" required maxlength="50">

    <label>Password</label>
    <input type="password" name="password" required>

    <button type="submit">Login</button>
</form>
<p>New user? <a href="{{ url_for('register') }}">Create an account</a></p>
{% endblock %}
```

templates/register.html

```
{% extends "base.html" %}
{% block content %}
<h2>Create Account</h2>
<form method="POST">
    <label>Username</label>
    <input type="text" name="username" required minlength="3" maxlength="50">

    <label>Password</label>
    <input type="password" name="password" required minlength="8">

    <button type="submit">Register</button>
</form>
<p>Already registered? <a href="{{ url_for('login') }}">Login</a></p>
{% endblock %}
```

templates/dashboard.html

```
{% extends "base.html" %}
{% block content %}
<h2>Welcome, {{ username }}!</h2>
<p>You are successfully logged in.</p>

<a class="button" href="{{ url_for('enable_2fa') }}">Enable 2FA</a>
<a class="button danger" href="{{ url_for('logout') }}">Logout</a>
{% endblock %}
```

templates/verify_2fa.html

```
{% extends "base.html" %}
{% block content %}
<h2>Two-Factor Authentication</h2>
<p>Enter the 6-digit code from your authenticator app.</p>
<form method="POST">
    <input type="text" name="code" inputmode="numeric"
           pattern="[0-9]{6}" maxlength="6" required>
    <button type="submit">Verify</button>
</form>
{% endblock %}
```

templates/enable_2fa.html

```
{% extends "base.html" %}
{% block content %}
<h2>Enable 2FA</h2>
<p>Add this secret to Google Authenticator, Microsoft Authenticator, or another TOTP app:</p>
<pre>{{ secret }}</pre>
<form method="POST">
    <input type="hidden" name="secret" value="{{ secret }}">
    <button type="submit">Confirm and Enable 2FA</button>
</form>
{% endblock %}
```

static/style.css

```
body {
    font-family: Arial, sans-serif;
    background: #f4f7fb;
    margin: 0;
    padding: 30px;
}

.container {
    max-width: 520px;
    margin: auto;
    background: white;
    padding: 30px;
    border-radius: 12px;
    box-shadow: 0 4px 18px rgba(0,0,0,0.08);
}

h1 { color: #173b70; }
h2 { color: #2457a6; }

label {
    display: block;
    margin-top: 14px;
    margin-bottom: 5px;
}

input {
    width: 100%;
    box-sizing: border-box;
    padding: 11px;
    border: 1px solid #bbb;
    border-radius: 6px;
}

button, .button {
    display: inline-block;
    margin-top: 18px;
    padding: 11px 16px;
    border: 0;
    border-radius: 6px;
    background: #2457a6;
    color: white;
    text-decoration: none;
    cursor: pointer;
}

.danger { background: #b42318; }

.message {
    padding: 10px;
    margin: 12px 0;
    border-radius: 6px;
}

.success { background: #dff7e8; color: #176b3a; }
.error { background: #fde2e2; color: #9b1c1c; }

pre {
    background: #eef2f7;
    padding: 12px;
```

```
    overflow-x: auto;
}
```

requirements.txt

```
Flask
argon2-cffi
pyotp
```

.gitignore

```
__pycache__/
*.pyc
.venv/
venv/
users.db
.env
```

README.md

```
# Secure Login System

A beginner-friendly Flask web application demonstrating secure user
registration, Argon2 password hashing, SQL-injection-safe database access,
sessions, logout, and optional TOTP-based two-factor authentication.

## Features
- User registration and login
- Argon2 password hashing
- Input validation
- Parameterized SQL queries
- Session management
- Logout
- Optional TOTP 2FA

## Setup

### 1. Install Python
Install Python 3.10+ and confirm:

```bash
python --version
```

2. Create the project folder
Create the files shown in this document.

3. Create a virtual environment

Windows:
```bash
python -m venv venv
venv\Scripts\activate
```

4. Install dependencies
```bash
pip install -r requirements.txt
```

5. Run
```bash
python app.py
```

Open:
http://127.0.0.1:5000

GitHub upload

```bash
git init
git add .
git commit -m "Initial secure login system"
git branch -M main
git remote add origin https://github.com/YOUR-USERNAME/secure-login-system.git
git push -u origin main
```

Security notes
For production, use HTTPS, a strong random SECRET_KEY stored in an
environment variable, secure cookie settings, CSRF protection, rate
limiting/lockout, secret storage for 2FA, and a production WSGI server.
```

3. How to Run the Project in VS Code

1. Install Python 3.10 or newer.

2. Open VS Code and create a folder named secure-login-system.

3. Create the files and folders exactly as shown in the project structure.

4. Open VS Code Terminal.

5. Run: python -m venv venv

6. Activate on Windows: venv\Scripts\activate

7. Install packages: pip install -r requirements.txt

8. Run the application: python app.py

9. Open http://127.0.0.1:5000 in your browser.

10. Register a user, log in, test logout, then optionally enable 2FA.

4. GitHub Upload — Step by Step

1. Create a new repository on GitHub, for example: secure-login-system.

2. Do not upload users.db, .env, venv, or Python cache files.

3. Open the project folder in VS Code.

4. Open Terminal and run: git init

5. Run: git add .

6. Run: git commit -m "Initial secure login system"

7. Run: git branch -M main

8. Copy your repository URL and run: git remote add origin YOUR_REPOSITORY_URL

9. Run: git push -u origin main

10. Refresh GitHub and verify that app.py, templates, static, README.md, requirements.txt, and .gitignore are present.

5. Security Explanation

• **Argon2:** Passwords are never stored as plain text. Argon2 produces a password hash that is stored in SQLite.

• **SQL injection:** User input is passed through parameterized queries such as SELECT ... WHERE username = ? rather than string concatenation.

• **Input validation:** Username length and password length are checked on the server.

• **Sessions:** After successful authentication, the user ID and username are stored in the Flask session.

• **Logout:** session.clear() removes the authentication state.

• **2FA:** pyotp generates and validates time-based one-time passwords.

• **Production warning:** The sample is educational. Production deployments should add HTTPS, CSRF protection, rate limiting, secure cookies, secret management, proper 2FA enrollment/QR provisioning, and a production server.

6. Expected Result

The completed application provides a secure-login demonstration matching the requested task: registration and login with hashed passwords, input validation and SQL-injection protection, session management with logout, and

optional 2FA. The project is structured so it can be uploaded to GitHub and demonstrated from VS Code.